Parcial práctico de Ingeniería de Software — La última transmisión

Integrantes: (nombre completo 1) y (nombre completo 2). **Versión entregada:** motor v1 + cambio de la hora 6 *(completar al recibirlo)*. Los diagramas, la matriz de trazabilidad y las capturas del tablero están en el *Anexo* (A1–A6, B, C).

1. Fundamentos

Problema. Un colectivo de escritores necesita publicar ficción interactiva sin depender de programadores: cargar relatos desde archivos, ofrecer opciones condicionadas por lo ocurrido y detectar errores del contenido antes de que lo juegue un lector.

Actores. *Escritora* (edita archivos JSON), *Lector* (juega), *Coordinadora del festival* (necesita saber qué corregir), *Diseñador de la experiencia* (define consecuencias). **Alcance:** motor local de consola con reglas booleanas, validación y reinicio. **Fuera de alcance:** cuentas, base de datos, guardado, red, editor visual, audio, imágenes, expresiones y retroceso.

Riesgos. (R1) *Contenido inválido que cierra el programa*: se mitiga validando todo al cargar y probando 17 variantes de error (RNF02, RNF03). (R2) *Acoplar el motor a la historia de ejemplo*: se mitiga evaluando condiciones desde los datos, con una segunda historia y una prueba que busca IDs de la historia en el código (RNF01).

Prioridad de calidad. 1.° modificabilidad del contenido (la escritora no programa), 2.° claridad de errores (la coordinadora debe saber qué corregir), 3.° confiabilidad (el lector nunca ve un cierre inesperado). Se priorizan sobre rendimiento, porque el volumen de datos es mínimo.

Responsabilidad profesional. Se reconoce la autoría de bibliotecas, de la historia (dada por el cliente) y de las ayudas de IA (sección 11). Solo se usan dependencias con licencia permisiva y se declara cada una. No se almacenan datos personales de lectores.

2. Proceso

Se eligió un **proceso incremental** con ciclos cortos: especificar → implementar → probar → ajustar. Alternativa considerada: **cascada**. Se descartó porque el cambio de la hora 6 es un requisito nuevo que llega con la solución en marcha; un proceso incremental absorbe ese cambio con menos retrabajo. *Especificación:* sección 4. *Desarrollo:* módulos pequeños con responsabilidades separadas. *Validación:* pruebas automatizadas y recorrido de los tres finales. *Evolución:* análisis de impacto del cambio (sección 9).

3. Agilidad

El backlog (docs/backlog.md) y el tablero (Anexo C) tienen columnas Pendiente / En curso / En revisión / Hecho y responsable por tarea. **Prácticas aplicadas:** (1) *backlog priorizado con entregas incrementales*: primero carga y juego de la historia de referencia, luego validación y la segunda historia; (2) *revisión entre integrantes* de requisitos y código antes de pasar a «Hecho». **Alternativa descartada:** Scrum con sprints formales y reuniones diarias, inviable en una ventana asincrónica de 12 horas. **Definición de terminado:** el elemento tiene prueba automatizada que pasa, fue revisado por el otro integrante, está en Git con mensaje descriptivo y la documentación afectada está actualizada. **Aprendizaje:** *(completar con lo vivido; ejemplo: «los requisitos con la palabra "válidas" generaron dudas»)*. **Acción de mejora:** *(completar)*.

4. Requisitos

Vocabulario común: *historia*, *escena*, *opción*, *condiciones*, *efectos*, *variable*, *sesión*, *final*, *habilitada* (opción cuyas condiciones se cumplen) y *bloqueo* (escena no final sin opciones habilitadas).

ID	N	Requisito	Criterio de aceptación
RF01	N01	Cargar una historia desde un archivo JSON	Archivo válido abre sesión; inexistente informa error sin sesión
RF02	N01	Iniciar en inicio con las variables iniciales	Inicio en E1, ambas variables en falso
RF03	N01	Cargar cualquier historia del formato sin cambiar el código	el_faro.json se juega sin tocar motor/
RF04	N02	Mostrar el texto de la escena actual	Tras elegir se muestra el texto del destino
RF05	N02	Mostrar solo las opciones habilitadas	CA01: en E3 con origen conocido, 2 opciones esperadas
RF06	N02	Habilitada = todas las condiciones se cumplen; mapa vacío habilita	En E3 hay exactamente dos opciones visibles
RF07	N03	Elegir aplica efectos y avanza al destino	«Investigar archivo» deja E3 con conoce_origen verdadero
RF08	N03	Rechazar opción inexistente o no habilitada sin alterar el	CA02: activar_codigo sin origen da error y estado igual

ID	N	Requisito	Criterio de aceptación
		estado	
RF09	N03	Entrada no válida en consola: aviso y nueva pregunta	«x», «0», «9» no cambian la escena
RF10	N04	Conservar las variables durante la partida	Efecto de E2 condiciona las opciones de E3
RF11	N05	Al llegar a un final, informarlo y no ofrecer opciones	E4, E5 y E6 muestran FIN
RF12	N05	Reiniciar restaura escena y variables iniciales	Tras reiniciar: E1 y variables en falso
RF13	N06	Rechazar archivo ilegible, estructura incorrecta o campos obligatorios ausentes	Mensaje con el campo; sin sesión
RF14	N06	Rechazar inicio inexistente e IDs de escena u opción repetidos	Mensaje con el ID
RF15	N06	Rechazar destinos inexistentes y variables no declaradas o valores no booleanos	CA03: destino E99 rechazado, nombra E99
RF16	N06	Rechazar finales con opciones y no finales sin opciones	Mensaje con la escena
RF17	N06	Informar todos los errores a la vez, con causa y campo o ID	Dos defectos producen al menos dos mensajes
RF18	N03	Escena no final sin opciones habilitadas: informar el bloqueo y permitir reiniciar	La consola avisa y ofrece reiniciar
RF19	N03	Sin retroceso en la versión 1	No existe comando de retroceso

Cobertura: N01→RF01–03; N02→RF04–06; N03→RF07–09, 18, 19; N04→RF10; N05→RF11–12; N06→RF13–17.

RNF	Medida	Procedimiento y evidencia
RNF01 Modificabilidad	Agregar o cambiar escenas, destinos o condiciones requiere editar 0 archivos de código	tests/test_rnf.py::test_rnf01_*: historia nueva y variante editada se juegan con el motor sin cambios; el motor no contiene IDs de la historia; git diff --stat de motor/ entre versiones
RNF02 Claridad de errores	100 % de los 17 casos inválidos nombran el ID o campo; 0 tracebacks	test_rnf02_*
RNF03 Confiabilidad	0 excepciones no controladas ante 17 archivos inválidos y 50 entradas inválidas; estado idéntico antes y después	test_rnf03_* (semilla fija)

Preguntas y supuestos. P1 ¿Variables no declaradas en condiciones o efectos? *Supuesto:* se rechaza al cargar. P2 ¿Entrada que no es opción? *Supuesto:* aviso y se repite la pregunta. P3 ¿condiciones y efectos obligatorios? *Supuesto:* opcionales; ausentes equivalen a mapas vacíos.

Conflicto retroceder vs. irreversible. La escritora pide retroceder siempre; el diseñador, decisiones irreversibles. *Pregunta al cliente:* ¿retroceso en todas las escenas, solo en las marcadas como reversibles, o ninguna; y si lo hay, ¿deshace efectos? *Acuerdo provisional:* la versión 1 no retrocede; reiniciar es la única vuelta, sin presuponer la decisión del cliente.

Defecto de requisitos hallado en la revisión cruzada. *(Sustituir por el hallazgo real de la pareja; ejemplo:)* RF05 decía «opciones válidas», ambiguo entre bien formadas en el archivo y disponibles en el estado actual. Se corrigió con el término *habilitada* y se separó validez del contenido (RF13–16) de disponibilidad en partida (RF05–06).

5. Modelado

Los diagramas están en el Anexo con el vocabulario del código. **Contexto (A1):** el sistema se relaciona con tres actores y los archivos de historia. **Casos de uso (A2):** cargar, validar (incluido por cargar), leer escena, elegir (incluye evaluar condiciones), reiniciar y ver errores. **Clases (A3):** Historia compone Escena y esta compone Opcion; Sesion mantiene el estado y lanza ErrorDeEleccion; el cargador crea el modelo y lanza HistoriaInvalida. **Secuencia (A4):** elegir una opción, con la rama de error. **Estados (A5):** una sesión está EnCurso, Finalizada o Bloqueada, y reiniciar vuelve a EnCurso. Si el archivo declara un inicio que ya es final, la sesión nace en Finalizada.

6. Arquitectura

El diagrama de componentes (A6) separa tres capas: **contenido** (historias/*.json), **motor** (motor/: modelo, validacion, cargador, sesion, errores) e **interfaz** (consola.py). La interfaz solo usa la API de Sesion y del cargador; el motor no imprime ni lee teclado; el contenido no contiene lógica. La separación se demuestra con el_faro.json: una historia distinta funciona sin editar motor/ (prueba test_segunda_historia_funciona_con_el_mismo_motor).

D1. Contenido declarativo en JSON, reglas interpretadas por el motor. *Contexto:* la escritora no programa y las reglas son solo igualdades booleanas. *Alternativas:* (a) historias como código Python; (b) un lenguaje de expresiones; (c) YAML. *Elección:* JSON con condiciones y efectos como mapas. *Consecuencias:* modificabilidad alta y cero dependencias; a cambio, las reglas no pueden expresar «o», comparaciones ni números sin cambiar el formato.

D2. Validar todo el contenido al cargar y devolver todos los errores. *Contexto:* la coordinadora debe saber qué corregir y el lector no debe ver cierres. *Alternativas:* (a) fallar en el primer error; (b) validar de forma perezosa durante la partida. *Elección:* una pasada de validación completa antes de crear la sesión. *Consecuencias:* los errores se corrigen de una vez y no hay sesiones con contenido roto; a cambio, no se detectan bloqueos que dependen del camino recorrido, que se tratan en ejecución (RF18).

7. Implementación

Python 3, biblioteca estándar y pytest. **Decisión de diseño detallado:** el modelo es inmutable (dataclass(frozen=True)) y todo el estado cambiante vive en Sesion, que devuelve copias de las variables. Así, ninguna interfaz puede alterar la partida saltándose elegir, lo que sostiene RF08 y baja el acoplamiento: la consola depende solo de métodos públicos. **Bibliotecas y licencias:** módulos json, argparse, dataclasses (biblioteca estándar, licencia PSF); pytest (MIT); para generar diagramas, Graphviz (EPL) y matplotlib (licencia basada en PSF), solo en desarrollo. Git versiona el trabajo; el historial está en docs/historial_git.txt.

8. Verificación

python -m pytest -v cubre U1–U5, I1, los criterios CA01–CA03, la consola y los tres RNF. El registro real de la última ejecución está en docs/resultados_pruebas.txt. Se recorrieron los tres finales y la segunda historia (grabación incluida en la demostración). **Fallos pendientes:** *(completar con el estado real al cierre; hasta ahora ninguno conocido)*.

9. Gestión del cambio

(Se completa al recibir la solicitud de la hora 6.) Debe registrar: versión de partida (etiqueta de Git), requisitos, modelos, componentes y pruebas afectados, ajustes al backlog, qué cambió y qué evidencia lo demuestra.

10. Trazabilidad

La matriz RF/RNF → modelos → componentes → evidencia está en el Anexo B, con rutas de archivos y nombres de pruebas.

11. Aportes, fuentes y herramientas de apoyo

Aportes por integrante *(completar con archivos, tareas y commits reales)*: (nombre 1): …; (nombre 2): ….

Herramientas de IA. Se usó un asistente de IA (Claude) para generar un primer borrador del motor, las pruebas, los diagramas y los textos de este informe. Cada resultado se verificó ejecutando las pruebas, recorriendo los finales a mano y revisando el código y los documentos entre ambos integrantes; lo no verificado se marcó como pendiente. *(Ajustar esta declaración a lo que realmente hicieron y a las reglas del docente.)* La historia de referencia es del cliente; la segunda historia, el_faro.json, fue creada para esta entrega. No se reutilizaron tutoriales ni fragmentos externos *(confirmar)*.